

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Selecting and Planning the Software Development Methodology for Your Own Project

Software Development Methodologies

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 4
Output	Development methodology plan, selection worksheets, milestone map, online justification, and group presentation
Maximum Score	80 points

Student / Project Information

Block: 3 - A	Date: 09-17-2026
System Title: SariwaBa?: Fish Freshness Classification System using Machine Learning and Mobile Computer Vision	Instructor: Jean Y. Villasin, MIT
Members: Marc Elson M. Rebato, Benedict A. Mendoza	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 4

1. Activity Alignment

and Purpose

Topic Learning Outcome: Compare common software development methodologies and select an appropriate approach for planning and managing the semester web-system project.

Course Outcome Alignment: Supports project planning and management by selecting and applying an appropriate development approach to the semester web-system project.

Purpose of the Activity

Activities 2 and 3 established the architecture and technology stack of your semester project. In Activity 4, your team will decide how the project will be planned, developed, reviewed, and delivered. You will compare common methodologies, analyze project conditions, select a defensible primary approach, and map it to SDLC phases, iterations or sprints, and semester milestones. The final plan should become the team's working guide for managing development during the remaining laboratory activities.

Important distinction: SDLC phases describe the major categories of development work, while a methodology or development approach organizes how those phases are planned and repeated. Agile approaches do not remove requirements, design, testing, or deployment; they arrange work in adaptive cycles. DevOps and CI/CD are complementary delivery practices and do not replace the selected development methodology.

2. Intended Learning Objectives

- Explain the SDLC phases and the purpose of using a software development methodology.
- Distinguish predictive, iterative, incremental, adaptive, and hybrid development approaches.
- Compare Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, and Rapid Application Development (RAD).
- Compare Agile approaches including Scrum, Kanban, and Extreme Programming (XP).
- Explain how DevOps and CI/CD complement development and delivery without replacing the project methodology.
- Evaluate requirement stability, risk, scope, schedule, team capability, and stakeholder involvement when selecting a methodology.
- Map the selected methodology to SDLC phases, iterations, sprints, project milestones, reviews, and semester deliverables.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the comparison, selection, milestone planning, and oral defense.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One project-specific Development Methodology Plan for the semester web-system project.

Activity Item	Requirement
Supporting Output	Methodology comparison matrix, scenario analysis, selection-criteria worksheet, SDLC mapping, semester milestone plan, DevOps/CI/CD delivery plan, 120-180 word methodology-selection justification, and group presentation.

Recommended Tools	Word/Google Docs, spreadsheet or timeline tool, diagrams.net/Draw.io, Canva or PowerPoint, GitHub Projects/Trello or an equivalent task board, and the assigned methodology case-study reading.
Presentation Format	5-7 minutes per group followed by 2-3 minutes of questions. Every member must explain an assigned selection criterion, project stage/iteration, or milestone.

4. Pre-Laboratory Requirements

- Approved system title and a brief statement of the community or organizational problem being addressed.
- Final or revised web-system architecture diagram from Laboratory Activity No. 2.
- Final or revised technology stack blueprint from Laboratory Activity No. 3.
- List of major system features, primary users, stakeholders, and any required approvals or feedback points.
- Known project constraints such as semester deadlines, technical risks, external services, uncertain requirements, and team limitations.
- The assigned methodology case-study reading and lecture notes on SDLC phases and software development methodologies.
- Access to the group GitHub repository and any current backlog, task board, calendar, or milestone list used by the team.

5. Part A - Development Approach Classification

Before comparing individual methodologies, classify the planning pattern correctly. Use the descriptions below to distinguish how a project responds to requirements, risk, delivery, and change. A specific methodology may combine more than one pattern.

Category	Meaning
SDLC phases	Planning, requirements/analysis, design, development, testing, deployment, and maintenance/review activities that every project must address in some form.
Predictive approach	Plans most scope, sequence, and major deliverables early; works best when requirements are relatively stable and change can be controlled.
Iterative approach	Repeats development cycles to refine the solution, requirements, design, or implementation using learning from earlier cycles.
Incremental approach	Builds and delivers the system in usable pieces or feature increments rather than waiting for one complete final release.
Adaptive / Agile approach	Uses frequent planning, feedback, and change response through short cycles or continuous flow.
Hybrid approach DevOps / CI/CD	Combines clearly defined practices from more than one approach because different parts of the project have different needs. Complementary delivery practices that support integration, testing, release, deployment, feedback, and operational reliability; they are not substitutes for the project methodology.

Methodology / Approach	Primary Planning Pattern	Strong Fit / Advantage	Limitation / Risk
Waterfall	Predictive and sequential	Appropriate when requirements, scope, technology, and approval procedures are clearly defined and unlikely to change. Its distinct phases make schedules, documentation, and phase approvals easy to understand.	Changes discovered after design or implementation can cause expensive rework. Working software and user feedback may arrive too late to expose incorrect assumptions.
V-Model	Predictive and verification-driven	Appropriate for systems requiring strong verification because every development stage is paired with a corresponding testing activity. It promotes early test planning, traceability, and clear acceptance criteria.	It is rigid when requirements change and can require substantial documentation and testing effort. It is less suitable when features must be explored through frequent prototypes.
Prototyping	Iterative and exploratory	Appropriate when users are uncertain about requirements, interfaces, or workflows. Early prototypes help stakeholders visualize the system and provide concrete feedback before full implementation.	Stakeholders may mistake a prototype for a finished product. Reusing poorly structured prototype code can produce maintainability, security, and performance problems.
Iterative	Iterative	Appropriate when requirements, design, or implementation must be refined through repeated development cycles and feedback.	Repeated revisions may cause rework, scope drift, and schedule uncertainty when iteration goals are not controlled.
Incremental	Incremental	Appropriate when the system can be divided into usable feature groups. It allows important features to be delivered and tested before the entire system is complete.	Increments require a sound overall architecture and careful integration planning. Poorly divided increments can create dependencies, inconsistent behavior, or repeated integration problems.
Spiral	Iterative and risk-driven	Appropriate for large, complex, or technically uncertain projects. Each cycle includes planning, risk analysis, engineering, and evaluation, allowing major risks to be investigated before further investment.	Formal risk analysis can be costly and difficult for a small or inexperienced team. The process may be excessive for a semester project with limited time and resources.
Rapid Application Development (RAD)	Iterative and incremental with rapid construction	Appropriate when an early working system is needed, requirements can be modularized, reusable components are available, and stakeholders can provide frequent feedback.	It depends on strong stakeholder availability, quick decisions, modular scope, and capable developers. Speed can weaken architecture, documentation, testing, or integration quality if poorly managed.

Scrum	Adaptive or Agile, iterative, and incremental	Appropriate for a small collaborative team working under a fixed schedule while requirements and technical knowledge continue to evolve. Time-boxed sprints, backlog prioritization, reviews, and retrospectives support frequent	Scrum requires disciplined backlog management, realistic sprint planning, team participation, and regular stakeholder feedback. Weak sprint goals or excessive mid-sprint changes can reduce predictability.
-------	---	---	--

		delivery and feedback.	
Kanban	Adaptive or Agile continuous flow	Appropriate when work arrives continuously or priorities change frequently. Visual workflow, explicit policies, and work-in-progress limits help expose bottlenecks and improve task flow.	Kanban does not automatically provide fixed iterations or scheduled review points. Without clear policies, deadlines, and completion criteria, work may remain unfinished or semester milestones may become difficult to predict.
Extreme Programming (XP)	Adaptive or Agile, iterative, and incremental	Appropriate when requirements change frequently and high software quality is essential. Practices such as test-driven development, continuous integration, refactoring, pair programming, and small releases encourage rapid feedback and reliable code.	XP requires strong technical discipline, close collaboration, frequent customer involvement, and sustained testing effort. Some practices may be difficult for a small student team to apply consistently.

6. Part B - Project Methodology Selection Criteria

Evaluate the conditions of your own project. Record evidence in the second column, then use the comparison cues to decide whether the condition favors a more predictive or a more iterative/adaptive approach. Your final choice should be based on the combined evidence, not on one criterion alone.

Choices should be based on the combined evidence, not on one criterion alone.				
Selection Criterion	Your Project Evidence / Condition	More Predictive Tends to Fit When...	More Iterative / Adaptive Tends to Fit When...	Team Interpretation
Requirement stability	The main features are defined, but interface details and classification requirements may change after testing and feedback. Requirements are known and unlikely to change.	Requirements are expected to evolve through feedback or discovery.	Requirements should be reviewed and refined between development cycles.	
Technical / project risk	Image preprocessing, ML integration, Google authentication, Flutter compatibility, and cloud deployment introduce uncertainty. Technology and integrations are familiar with few unknowns.	High uncertainty, new technology, risky integrations, or research-heavy work require learning cycles.	Risky integrations should be tested early instead of waiting until final development.	

Scope clarity	The major features and system boundaries are established, but detailed acceptance criteria and ML behavior still require validation. Major features and boundaries can be defined early.	Feature priorities or details may change as prototypes and feedback are reviewed.	The system should be divided into manageable feature increments with clear acceptance criteria.
Schedule / deadline	The project has a fixed semester deadline and must demonstrate measurable progress regularly. Formal phase gates and fixed planned deliverables are practical.	Frequent usable increments or short time-boxes are needed before the final deadline.	Short sprints will help prioritize essential features and produce measurable deliverables.
Team size and experience	Planning assumption: SariwaBa is handled by a small student team with different levels of Flutter, Django, database, and ML experience. Roles can be specialized and work can follow a planned sequence.	A small cross-functional team can collaborate frequently and adjust tasks quickly.	Members should collaborate, share knowledge, and adjust assignments according to progress and difficulty.

Stakeholder involvement	The instructor and adviser can review outputs. Consumers and vendors are the intended users whose feedback may improve usability. Stakeholders are available mainly at defined approval points.	Stakeholders can provide frequent feedback, review prototypes, or reprioritize work.	Feedback should be collected during scheduled sprint reviews and used to update priorities.
Need for change and feedback	Testing may reveal issues involving image quality, classification results, API behavior, and user workflow. Changes are rare, costly, or tightly controlled.	Learning, testing, and user feedback are expected to change the next iteration or work queue.	Changes should be evaluated after each sprint and placed in the backlog without unnecessarily disrupting active work.

Selection rule

Choose a methodology because it fits the project conditions, not because it is popular or familiar. A hybrid approach is acceptable only when the team identifies which practices are combined and where each one applies. DevOps and CI/CD may complement any selected methodology but should not be presented as the methodology itself.

7. Part C - Methodology Scenario-Analysis Workshop

For each scenario, identify a defensible methodology or approach and explain the tradeoff. More than one answer may be acceptable when the reasoning is supported by the project conditions.

Scenario	Key Project Condition	Best-Fit Methodology	Why It Fits	Main Risk / Limitation	Alternative / Hybrid Option
1. Government registration module	Requirements are defined by fixed forms, rules, and approval steps.	Waterfall	The requirements and sequence can be defined before development, making planned phases and formal approvals practical.	A change in government rules may require revisiting completed phases.	V-Model may be used when every requirement needs a corresponding verification test.
2. New citizen reporting interface	Users are unsure what screen flow will be easiest until they see a working prototype.	Prototyping	A prototype allows users to evaluate the interface and provide feedback before full implementation.	Users may mistake the prototype for a finished system, while repeated revisions may increase development time.	Iterative development can refine the interface through repeated working versions.
3. Verification intensive transaction module	Each development stage requires a corresponding test or verification activity.	V-Model	The V-Model pairs each development stage with a related testing stage, supporting traceability and early test planning.	It is rigid and may require substantial rework when requirements change.	Waterfall with detailed testing and formal phase gates may be used when requirements are completely stable.
4. Semester project with frequent adviser feedback	A small team must show working features regularly and reprioritize based on reviews.	Scrum	Time-boxed sprints allow the team to plan, develop, test, demonstrate, and improve features regularly.	Scrum requires disciplined backlog management, realistic sprint planning, and regular participation.	Incremental development may be used if the team needs feature releases without complete Scrum events and roles.

5. Short deadline for an early working demo	The team needs rapid construction, reusable components, and strong stakeholder availability.	Rapid Application Development	RAD emphasizes fast prototyping, reusable components, and continuous stakeholder participation to produce an early working system.	Speed may reduce documentation, architecture, or testing quality if the process is poorly controlled.	Scrum with short sprints and prototyping can provide rapid progress with stronger planning and review points.
6. GIS/ML feature with uncertain integration risk	External APIs, data quality, or model behavior may create technical uncertainty that must be evaluated repeatedly.	Spiral	Spiral development places risk analysis and experimentation at the center of every cycle, making it suitable for uncertain technical work.	Its formal risk-analysis process may be too costly or complex for a small semester project.	Scrum can include technical prototypes or research tasks during early sprints to reduce integration risks.

Scenario-analysis checklist:

- Use evidence from requirement stability, risk, scope, schedule, team capability, and stakeholder involvement.
- Do not select a methodology only because the team already knows it or because it is currently popular.
- Distinguish the development methodology from tools such as Trello/GitHub Projects and from complementary DevOps/CI/CD practices.

- State at least one tradeoff, limitation, or alternative approach for every scenario.

8. Part D - Select and Map Your Project Methodology

Selected primary methodology: Scrum

Complementary approach / practice(s):

Prototyping, selected Extreme Programming engineering practices, and DevOps/CI/CD

Why this choice fits the project (3-5 sentences):

Scrum fits SariwaBa because the project has a fixed semester deadline but still contains uncertainty involving image processing, ML integration, authentication, and deployment. Its time-boxed sprints allow the team to develop and test the system as a series of working increments. Sprint reviews provide regular opportunities for feedback from the instructor, adviser, and intended users.

Project Methodology and SDLC Mapping Worksheet

Map the major SDLC activities to the way your selected methodology will actually organize the semester project. If the methodology repeats phases, show where the repetition occurs. If it uses sprints or continuous flow, identify the planning and review points clearly.

SDLC Phase / Activity	How It Is Handled	Time-Box / Duration	Iteration / Sprint / Stage	Main Team Role(s)	Stakeholder Check	Deliverable / Exit Evidence
Project planning	Establish the product goal, prioritize the backlog, identify risks, and define sprint goals.	Sprint 0 and Sprint Planning for Sprints 1–5	Week 1, followed by a short planning session at the start of each sprint	Product Owner, Scrum Master, Developers	Instructor or adviser reviews the project direction.	Prioritized product backlog, risk list, and approved sprint goal
Requirements / analysis	Convert features into backlog items with acceptance criteria. Refine requirements using feedback and test results.	Sprint 0 and backlog refinement during Sprints 1–5	Initial analysis during Week 1, then refinement before or during each sprint	Product Owner and Developers	Instructor, adviser, or intended users clarify requirements when needed.	Reviewed backlog items and acceptance criteria
Design	Preserve the architecture while preparing detailed interface, API, database, and component designs for selected backlog items.	Initial design during Week 1, followed by early design work within each sprint	Sprint 0 for overall design and Sprints 1–5 for feature-level design	Developers and assigned UI or system designer	Relevant designs are reviewed before implementation.	Wireframes, API specifications, and updated technical diagrams
Development / implementation	Build the selected Flutter, Django, PostgreSQL, authentication, and ML-integration features.	Most of each two-week sprint	Development stage of Sprints 1–5	Developers	Working features are demonstrated during the sprint review.	Working code committed through reviewed pull requests

Testing / verification	Perform unit, API, database, interface, integration, and acceptance testing within the same sprint as development.	Continuous throughout each sprint, with final verification before the review	Testing stage within Sprints 1–5	Developers and assigned testers	Stakeholders confirm whether the increment meets its acceptance criteria.	Test results, resolved critical defects, and accepted increment
Integration / deployment	Integrate approved code, apply Django migrations, and deploy stable builds to the appropriate test or production environment.	Near the end of each sprint; final deployment during Weeks 10–11	Integration stage of every sprint and production deployment during Sprint 5	Developers and deployment coordinator	Instructor or adviser reviews deployed increments when required.	Successful build, migration record, and accessible deployment
Review / feedback	Demonstrate the completed increment, record feedback, and conduct a	End of each two-week sprint	Sprint Review and Sprint Retrospective for Sprints 1–5	Entire Scrum team	Instructor, adviser, and available intended users review the increment.	Sprint review notes, accepted work, feedback, and retrospective

	retrospective.					actions
Maintenance / improvement	Record defects, improvements, and requirement changes in the backlog for prioritization in a later sprint.	Backlog update after each sprint and final release stage	After every sprint review and during Week 12 finalization	Product Owner and Developers	Stakeholders verify important corrections or improvements.	Updated backlog, corrected release, and maintenance record

SDLC mapping guidance

Waterfall and V-Model usually emphasize planned phases and formal transitions. Prototyping, Iterative, Incremental, Spiral, and RAD repeat or accelerate selected activities. Scrum normally organizes work into sprints; Kanban uses continuous flow and work-in-progress control; XP emphasizes short feedback cycles and engineering practices. Your mapping must reflect the actual method selected by the team.

9. Part E - Semester Project Milestone Plan

Translate the selected methodology into a realistic semester schedule. Every milestone should have a measurable output and a review, approval, demonstration, test, or feedback point. Use your actual course deadlines when they are already known.

Week(s) / Date	Phase / Sprint / Iteration / Milestone	Planned Features / Scope	Key Activities	Evidence / Deliverable	Review / Approval / Feedback
Week 1 / September 7–13, 2026	Sprint 0: Project foundation	Requirements, backlog, environments, repository structure, wireframes, and technical risks	Confirm the architecture and stack, create backlog items, define acceptance criteria, assign roles, and prepare development environments.	Prioritized backlog, role assignments, wireframes, risk list, and working project setup	Team, instructor, or adviser reviews the development plan and priorities.
Week 2-3 / September 14–27, 2026 Week 4-5 / September 28–October 11, 2026	Sprint 1: Authentication and user foundation Sprint 2: Classification workflow	Flutter application foundation, Google Sign-In, backend token verification, and user profile Image capture or upload, client validation, classification API, server validation, and initial result interface	Implement the application structure, authentication flow, protected API access, and basic profile functions. Implement multipart upload, Bearer-token requests, image checking, preprocessing pipeline, and temporary or initial classification response.	Working sign-in flow, protected endpoint tests, and profile demonstration Demonstrable image submission and classification result flow	Sprint review verifies that authenticated users can securely access protected resources. Review the image workflow, error messages, usability, and API behavior.

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 4

Week(s) / Date	Phase / Sprint / Iteration / Milestone	Planned Features / Scope	Key Activities	Evidence / Deliverable	Review / Approval / Feedback
Week 6-7 / October 12–25, 2026	Sprint 3: ML and history integration	External ML-service integration, classification persistence, fish images, and history endpoints	Connect the backend to the ML service, save classification records through Django ORM, and implement history functions.	End-to-end classification with label, confidence, scan ID, and saved history	Review classification behavior, stored data, history accuracy, and failure handling.
Week 8-9 / October 26–November 8, 2026	Sprint 4: Quality and interface improvement	History interface, profile refinement, validation, security, and cross-platform behavior	Improve the Flutter interface, complete history functions, strengthen error handling, and perform integration and security testing.	Tested Flutter Web and Mobile increment with resolved priority defects	Conduct usability review and confirm acceptance criteria for core features.
Week 10-11 / November 9–22, 2026	Sprint 5: Deployment and acceptance	Vercel deployment, Render services, Render PostgreSQL, Android APK, and acceptance testing	Configure production environments, deploy services, conduct end-to-end testing, and prepare release documentation.	Accessible web system, running backend and database, Android APK, and acceptance-test results	Instructor, adviser, and available users review the release candidate.

Week 12 / November 23–29, 2026	Final release and presentation	Final corrections, documentation, release evidence, and presentation	Resolve critical defects, confirm deployment, complete documentation, and rehearse the oral defense.	Final SariwaBa release, methodology artifacts, test evidence, and presentation	Final team review and submission approval
--------------------------------------	-----------------------------------	---	---	---	---

Milestone planning rule

The methodology plan must be visible in the schedule. A sprint-based plan should show sprint goals and reviews; an incremental plan should show usable feature increments; a predictive plan should show phase gates and approval points; a Kanban-style plan should show how work enters, moves through, and exits the workflow. Do not use vague milestones such as “continue coding” without an observable deliverable.

10. Part F - DevOps and CI/CD Complementary Delivery Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Version-control workflow	How branches, commits, merges, and code reviews support the selected methodology.	The team will use Git and GitHub. Development work will use short feature branches, descriptive commits, pull requests, and code review before integration into the main branch. The main branch should contain only reviewed and working code.
Continuous integration	When code is integrated and what build/check should run before changes are accepted.	Planning-stage decision: GitHub Actions will run automated checks for pull requests and updates to the main branch. Checks should include Flutter formatting, analysis, and tests, together with backend tests and migration checks. Code will not be integrated when required checks fail.
Testing practice	What unit, API, database, UI, or acceptance checks will be performed and when.	Testing will occur during every sprint. It will include Flutter unit or widget tests, Django and API tests, authentication and authorization checks, PostgreSQL integration tests, ML-service integration tests, and end-to-end acceptance tests for the classification flow.
Database change control	How PostgreSQL schema changes, migrations, and seed data will be coordinated with application development.	PostgreSQL schema changes will be created and tracked using Django migrations. Migration files will be reviewed with the related application code. The team will avoid unrecorded manual production-schema changes and will test migrations before deployment.

Deployment stages	Local, test/staging, and final deployment targets; identify any approval before release.	Development begins locally, followed by preview or test deployments and then final production deployment. Flutter Web will use Vercel, Django and the ML service will use Render Web Services, PostgreSQL will use Render PostgreSQL, and the mobile application will be distributed as an Android APK.
Environment configuration	How API URLs, credentials, and other environment specific values are kept outside source code.	API URLs, Google OAuth settings, database credentials, secret keys, and other environment-specific values will be stored in secure environment variables. Credentials and secret configuration files must not be committed to GitHub.
Monitoring / feedback	How errors, defects, stakeholder feedback, and completed work are recorded and fed into the next cycle or milestone.	The team will review deployment logs, backend errors, failed API requests, test failures, and stakeholder feedback. Defects and improvements will be recorded as backlog items and prioritized for the next appropriate sprint. Sensitive tokens and uploaded-image data must not be exposed in logs.

Release / rollback	How the team will identify a stable release and recover from a failed deployment or defective change.	Each release will be linked to a stable Git commit or version tag. If a deployment fails, the team will restore or redeploy the previous stable version. Database migrations will be tested carefully, and backups or recovery steps will be prepared before risky production changes.
---------------------------	---	--

Recommended minimum methodology evidence in the project repository or shared workspace (adapt as needed):

```
docs/methodology/
  development-methodology-plan.pdf
  methodology-selection-justification.md
```

```
milestone-plan/
sprint-or-iteration-notes/
review-and-feedback-records/
test-evidence/
release-notes/
README.md - brief description of the team's development workflow
```

11. Part G - Construct the Development Methodology Plan

Create one clear project-specific methodology plan that explains how your team will manage the semester project from requirements through delivery. The plan may be presented as a structured one-page or multi-page visual, timeline, workflow, or combination of diagram and schedule, but it must remain consistent with the completed worksheets.

No.	Required Element	What Must Be Shown
1	Project identity	Official system title, project problem, major users/stakeholders, and team members.
2	Selected primary methodology	Exact methodology/approach selected and whether it is predictive, iterative, incremental, adaptive, or hybrid.
3	Selection rationale	Project evidence from requirement stability, risk, scope, schedule, team, and stakeholder involvement.
4	SDLC mapping	How planning, requirements, design, development, testing, deployment, and review are organized.
5	Iteration / sprint / flow structure	Length or sequence of iterations/sprints/stages, or the workflow used if the method does not use fixed sprints.
6	Requirements / backlog control	How features, tasks, priorities, changes, and acceptance criteria will be recorded and updated.
7	Team responsibilities	Who plans, develops, tests, reviews, documents, coordinates, or approves work.
8	Semester milestones	Major target dates or weeks, expected deliverables, demonstrations, and completion criteria.
9	Stakeholder involvement	When users, instructor, adviser, client, or other stakeholders provide feedback or approval.
10	Quality and testing	When testing occurs and how defects, rework, verification, or acceptance are handled.

11	DevOps / CI/CD support	Git/GitHub workflow, integration, build/test, database change control, deployment, and feedback practices.
12	Adaptation / control rule	How the team responds when requirements, risks, schedule, or technical conditions change.

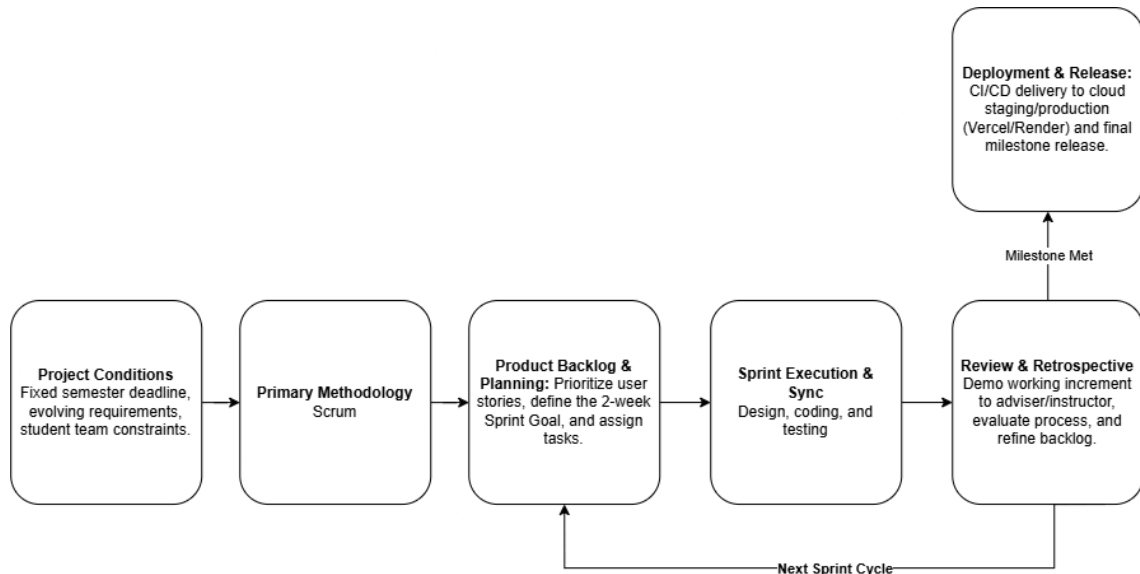
Minimum Plan Rule

Your plan must show a complete management path from project requirements or backlog -> planned phase/iteration/sprint/work queue -> development and testing -> review/feedback -> milestone or release. If the selected method is iterative or adaptive, show how the cycle repeats. If it is predictive, show phase gates. If it is hybrid, label exactly where each approach is used.

12. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the decision-and-planning logic expected in the final output. Replace the generic labels with the actual constraints, methodology, iteration/stage structure, milestones, stakeholder reviews, and delivery practices of your own project.

Figure 1.



Reference methodology selection and semester planning flow.

Step	Reference Flow
1	Identify project conditions: requirement stability, risk, scope, schedule, team, and stakeholder involvement.
2	Compare predictive, iterative, incremental, adaptive, and hybrid options using the actual project evidence.
3	Select one primary methodology and identify any complementary practices such as prototyping, DevOps, or CI/CD.

4	Map SDLC phases to stages, iterations, sprints, or continuous workflow according to the selected methodology.
5	Define semester milestones, deliverables, testing points, demonstrations, reviews, and stakeholder feedback.
6	Use results and feedback to continue the planned next stage or adapt the next cycle when the selected methodology permits it.

13. Part H - Methodology-to-Project Trace

Selected feature / work package:

Fish Image Classification and History Workflow

Target milestone / release: Sprint 2 initial classification workflow and Sprint 3 complete ML and history integration

Step / Iteration	Planned Work	Methodology Principle / Practice	Evidence / Deliverable	Review / Feedback

Backlog preparation	Define the classification workflow, requirements, and acceptance criteria.	Product backlog refinement	Prioritized user story and acceptance criteria	Team reviews scope and technical risks.
Sprint 2 planning and design	Plan the Flutter upload interface, API request, validation, and result display.	Sprint planning and feature-level design	Sprint goal, assigned tasks, and interface or API design	Team confirms that the work fits the sprint.
Sprint 2 implementation	Implement image capture or upload, client validation, Dio request, Django authentication, server validation, and preprocessing.	Incremental development and continuous testing	Working initial classification workflow	Demonstrate image submission and result display.
Sprint 3 integration	Connect the external ML service and return the label, confidence, and scan ID.	Iterative development and risk reduction	End-to-end ML classification response	Review classification behavior and error handling.
Sprint 3 persistence	Save classification and image information through Django ORM and display user history.	Working software increment	Stored classification record and updated history	Verify that users can access only their own records.
Testing and review	Perform API, database, authentication, integration, and acceptance tests.	Definition of Done and Sprint Review	Test results and accepted working increment	Record feedback and identified bugs.
Backlog update	Prioritize corrections and improvements for the next sprint.	Retrospective and adaptation	Updated product backlog	Approved changes enter the next sprint cycle.

14. Part I - Methodology-Selection Justification

Write 120-180 words explaining why the selected development methodology is appropriate for your own semester project. This will also serve as the basis of the online methodology-selection justification. Address all of the following:

- Name the selected primary methodology and classify it as predictive, iterative, incremental, adaptive, or hybrid as appropriate.
- Use project evidence related to requirement stability, risk, scope, schedule, team capability, and stakeholder involvement.
- Explain why the selected methodology is more appropriate than at least two reasonable alternatives. • Describe how SDLC phases, iterations, sprints, work flow, and semester milestones will be organized. • Explain how DevOps and CI/CD practices will complement development and delivery, including one limitation or risk the team must manage.

SariwaBa will use Scrum as its primary methodology. Scrum is an adaptive Agile framework that supports iterative and incremental development. The project's main scope is established, but technical details may change as the team tests image preprocessing, Google authentication, the external ML service, Flutter compatibility, and cloud deployment. Its fixed semester deadline, small team, and need for instructor, adviser, and user feedback favor short, time-boxed sprints. Waterfall is less suitable because problems

discovered late could require major rework. Spiral addresses technical risk but requires formal risk analysis that is excessive for a semester project. The SDLC activities will be repeated across Sprint 0 and five development sprints, with each sprint including planning, requirements, design, implementation, testing, integration, and review. GitHub branches, code reviews, automated checks, Django migrations, and staged Vercel and Render deployments will support delivery. The team must control scope by protecting the active sprint and placing noncritical changes in the prioritized product backlog.

15. Final Quality Checklist

Checklist A	Checklist B
☐ The official project title, users, features, and real project conditions are used.	☐ The selected primary methodology is clearly named and correctly classified.
☐ Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP were compared.	☐ Selection criteria use actual evidence about requirements, risk, scope, schedule, team, and stakeholders.
☐ Scenario analysis includes a methodology choice, reasoning, tradeoff, and alternative.	☐ The team explains why at least two reasonable alternative methodologies were not selected.
☐ SDLC phases are mapped consistently to the selected methodology.	☐ Iterations, sprints, stages, or continuous workflow are represented accurately.
☐ The semester milestone plan includes dates/weeks, deliverables, and review or feedback points.	☐ Stakeholder involvement and change-management points are visible in the plan.
☐ DevOps and CI/CD are treated as complementary delivery practices, not as the primary methodology.	☐ The plan is feasible for the team size, available time, technical risks, and semester deadline.
☐ The written methodology-selection justification agrees with all worksheets and the final plan.	☐ Every group member can explain the methodology, milestone sequence, and major tradeoffs without reading a script.

16. Submission Requirements

- PDF of the final Development Methodology Plan in a clear, readable format.
- Completed Parts A-H and the 120-180 word methodology-selection justification.
- Export or screenshot of the semester milestone timeline, sprint/iteration plan, or task board used to support the final plan.
- Updated project GitHub repository or shared project workspace containing the methodology artifacts under a folder such as docs/methodology/.

- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA4_SystemName.
- Deliver a 5-7 minute group presentation using the final methodology plan as the main visual, followed by 2-3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own project conditions and planning decisions. All tools may be used to clarify methodology concepts or review ideas, but the final comparison, selection criteria, methodology choice, SDLC mapping, milestone plan, and written justification must be checked, understood, and defended by the team. The instructor may ask any member to explain why a methodology was selected or trace how one feature will move through the planned process without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unclear reasoning, unequal participation, excessive reading, inconsistent milestone mapping, or inability to defend the selected methodology against reasonable alternatives.

Criterion	Evidence for Full Credit	Points
Methodology comparison accuracy	The comparison correctly distinguishes the major characteristics, strengths, limitations, and appropriate use of Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP.	10
Scenario analysis and tradeoff reasoning	Each scenario uses relevant project conditions to select a defensible method, explains why it fits, identifies a limitation, and considers a reasonable alternative or hybrid option.	8
Project-specific selection criteria	Requirement stability, risk, scope, schedule, team capability, stakeholder involvement, and need for change/feedback are evaluated using evidence from the actual project.	10
Selected methodology and written justification	The primary methodology is clearly named, correctly classified, supported by project evidence, compared with alternatives, and justified in a coherent 120-180 word explanation.	12

SDLC, iteration, sprint, or flow mapping	Planning, requirements, design, development, testing, deployment, review, and improvement are mapped accurately to the selected method and its stages, cycles, sprints, or continuous workflow.	12
Semester milestone plan	The schedule is realistic, project-specific, and aligned with the selected methodology; milestones contain measurable scope, deliverables, review/feedback points, and completion evidence.	12
DevOps and CI/CD complementary delivery plan	Version control, integration, testing, database change control, deployment, configuration, monitoring/feedback, and release/rollback practices appropriately support the selected methodology.	6
Group presentation and oral defense	The presentation is clear, organized, and within time; all members participate and correctly defend the methodology choice, project mapping, milestones, tradeoffs, and delivery practices.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which project selection criterion had the greatest influence on your methodology choice, and why?

Technical and project risk had the greatest influence. SariwaBa depends on image preprocessing, Google authentication, an external ML service, Flutter, Django, PostgreSQL, and cloud deployment. Scrum allows these risks to be tested and addressed through repeated sprints.

Which reasonable alternative methodology did your group reject, and what project condition made it less suitable?

The group rejected Waterfall because SariwaBa may require changes after integration testing and stakeholder feedback. Discovering these issues near the end of a sequential process could cause significant rework.

Where will the most important iteration, sprint review, phase gate, or workflow review occur in your semester plan?

The most important review will occur at the end of Sprint 3. At this point, the team should demonstrate the complete classification flow, including ML integration, confidence results, database storage, and classification history.

How will stakeholder feedback or requirement changes be handled without losing control of scope and schedule?

Feedback and change requests will be recorded in the product backlog. The Product Owner will prioritize them for a future sprint. Noncritical changes will not interrupt the active sprint.

Which DevOps or CI/CD practice should your team establish first, and what evidence will show that it is working?

The team should first establish automated checks for pull requests. It is working when tests run consistently, failed checks prevent incorrect code from being merged, and fewer integration problems appear in the main branch.